

Marquesina — Refactor Inicial y Organización Arquitectónica

Visión General

Marquesina ya no es únicamente una interfaz que desplaza texto. Comienza a comportarse como un pequeño sistema de transmisión:

```
Fuentes de texto
↓
Preprocesamiento Bash
↓
Archivo consolidado
↓
Carga Java
↓
Render Swing
↓
Animación temporal
```

La idea de esta reorganización es:

- separar responsabilidades,
- facilitar mantenimiento,
- preparar crecimiento futuro,
- mantener estabilidad de `main`,
- y convertir el proyecto en un laboratorio pedagógico.

Estructura Recomendada

```
Marquesina/
|
├─ src/
|   ├── Main.java
|   ├── TextLoader.java
|   ├── RenderUtils.java
|   └── Config.java
|
├─ scripts/
|   └── mezclar_marquesina.sh
```

```

|
|— lib/
|   |— txt/
|       |— marquesina_mix.txt
|
|— README.md
|— .gitignore
|— .vscode/

```

Main.java

```

import javax.swing.*;
import java.awt.*;

/*
=====
MARQUESINA
=====
Motor visual principal.

Responsabilidades:
- crear ventana,
- administrar animación,
- coordinar recarga textual,
- renderizar transmisión.

POO:
Main extiende JPanel.
Esto significa que Main ES un panel gráfico.

Herencia:
Permite reutilizar comportamiento gráfico de Swing.
=====
*/

public class Main extends JPanel {

    /*
    =====
    CONFIGURACIÓN GENERAL
    =====
    Config concentra rutas y parámetros importantes.

    Centralizar configuración evita:

```

```

- números mágicos,
- rutas repetidas,
- mantenimiento caótico.
=====
*/

private String mensaje =
    TextLoader.cargarTexto(Config.RUTA_NOTAS);

// Posición horizontal inicial
private int x = 800;

/*
=====
CONSTRUCTOR PRINCIPAL
=====
Inicializa:
- fondo,
- timers,
- velocidad,
- recarga.

El constructor representa el momento de nacimiento
del objeto.

Mundo Humano:
Un sistema también necesita memoria inicial,
ritmo y límites antes de comenzar a moverse.
=====
*/

public Main() {

    setBackground(Color.BLACK);

    // Reduce parpadeo visual
    setDoubleBuffered(true);

    iniciarRecargaTexto();
    iniciarAnimacion();
}

/*
=====
ANIMACIÓN PRINCIPAL
=====
Timer de Swing:
ejecuta código repetidamente.

```

IMPORTANTE:

Swing posee su propio hilo gráfico.

Por eso Swing Timer es más seguro para UI.

```
=====
*/

private void iniciarAnimacion() {

    Timer timer = new Timer(
        Config.INTERVALO_FRAME,
        e -> {

            x -= Config.VELOCIDAD;

            if (x < -(mensaje.length() * 14) - 300) {

                x = getWidth();
            }

            repaint();
        }
    );

    timer.start();
}

/*
=====
RECARGA DE TRANSMISIÓN
=====
Recarga el archivo consolidado.

Actualmente:
- Bash prepara transmisión.
- Java consume transmisión.

Separar procesamiento y render simplifica el sistema.
=====
*/

private void iniciarRecargaTexto() {

    int tiempoRecarga =
        RenderUtils.calcularTiempoRecarga(mensaje);

    System.out.println(
        "Tiempo estimado ciclo: "
    );
}
```

```

        + tiempoRecarga / 1000
        + " segundos"
    );

    Timer recargaNotas =
        new Timer(tiempoRecarga, e -> {

            System.out.println(
                "Recargando transmisión..."
            );

            mensaje =
                TextLoader.cargarTexto(
                    Config.RUTA_NOTAS
                );

        });

    recargaNotas.start();
}

/*
=====
RENDER
=====
paintComponent es invocado constantemente por Swing.

Aquí solamente debe existir lógica visual.

Regla importante:
Nunca mezclar carga de archivos,
networking o procesos pesados aquí.
=====
*/

@Override
protected void paintComponent(Graphics g) {

    super.paintComponent(g);

    Graphics2D g2d = (Graphics2D) g;

    RenderUtils.activarAntialias(g2d);

    g.setFont(
        new Font(
            "SansSerif",
            Font.BOLD,
            28

```

```

        )

    };

    g.setColor(Color.GREEN);

    g.drawString(mensaje, x, 50);
}

/*
=====
MAIN
=====
Punto de entrada del programa.

 Toda aplicación Java comienza aquí.
=====
*/

public static void main(String[] args) {

    JFrame ventana = new JFrame("Marquesina");

    Main panel = new Main();

    ventana.add(panel);

    ventana.setSize(
        Toolkit
            .getDefaultToolkit()
            .getScreenSize()
            .width,
        100
    );

    ventana.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE
    );

    ventana.setLocationRelativeTo(null);

    ventana.setVisible(true);
}
}

```

TextLoader.java

```
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;

/*
=====
TEXT LOADER
=====
Responsabilidad única:
leer transmisión textual.

P00:
Clase utilitaria.

No necesita crear objetos.
Por eso usa métodos static.
=====
*/

public class TextLoader {

    /*
    =====
    CARGA TEXTO DESDE ARCHIVO
    =====
    Lee archivo consolidado generado por Bash.

    Mundo Humano:
    Toda transmisión depende de memoria persistente.
    El archivo funciona como una huella temporal
    entre procesos distintos.
    =====
    */

    public static String cargarTexto(String rutaArchivo) {

        try {

            File archivo = new File(rutaArchivo);

            return Files.readString(archivo.toPath());

        } catch (IOException e) {
```

```

        System.out.println(
            "Error leyendo transmisión."
        );

        return "No se pudo cargar la transmisión.";
    }
}
}

```

RenderUtils.java

```

import java.awt.*;

/*
=====
RENDER UTILS
=====
Métodos auxiliares de render.

Separar utilidades evita:
- duplicación,
- contaminación visual,
- clases gigantes.
=====
*/

public class RenderUtils {

    /*
    =====
    SUAVIZADO TIPOGRÁFICO
    =====
    Anti alias reduce bordes irregulares.
    =====
    */

    public static void activarAntialias(Graphics2D g2d) {

        g2d.setRenderingHint(
            RenderingHints.KEY_TEXT_ANTIALIASING,
            RenderingHints.VALUE_TEXT_ANTIALIAS_ON
        );
    }
}

```



```

/*
=====
CÁLCULO DE RECORRIDO
=====
Estima duración total del desplazamiento.

Matemáticamente:

distancia / velocidad = tiempo

Mundo Humano:
Toda transmisión necesita duración.
Incluso la información posee una velocidad
de desaparición.
=====
*/

public static int calcularTiempoRecarga(
    String mensaje
) {

    int anchoPantalla =
        Toolkit
            .getDefaultToolkit()
            .getScreenSize()
            .width;

    int anchoTexto = mensaje.length() * 14;

    int distanciaTotal =
        anchoPantalla + anchoTexto;

    return (
        distanciaTotal
        / Config.VELOCIDAD
    ) * Config.INTERVALO_FRAME;
}
}

```

Config.java

```

/*
=====
CONFIG

```

```

=====
Configuración central del sistema.

Buena práctica:
concentrar constantes globales.
=====
*/

public class Config {

    // Ruta archivo consolidado
    public static final String RUTA_NOTAS =
        "/home/remi/Documents/Dispensación/"
        + "Senda de Java/Marquesina/lib/txt/"
        + "marquesina_mix.txt";

    // Movimiento
    public static final int VELOCIDAD = 2;

    // 16 ms ≈ 60 FPS
    public static final int INTERVALO_FRAME = 16;
}

```

mezclar_marquesina.sh

```

#!/bin/bash

# =====
# MARQUESINA MIXER
# =====
# Pipeline textual previo al runtime.
#
# Bash:
# excelente para automatización de archivos.
#
# Responsabilidad:
# - unir textos,
# - mezclarlos,
# - generar transmisión consolidada.
# =====

# =====
# ORIGEN DE NOTAS
# =====

```

```

ORIGEN="/home/remi/Documents/Texto/MarquesinaNotas"

# =====
# ARCHIVO FINAL
# =====

SALIDA="$ORIGEN/marquesina_mix.txt"

# =====
# LIMPIA SALIDA ANTERIOR
# =====
# El operador > vacía el archivo.
#
# Mundo Humano:
# Algunas transmisiones necesitan olvidar
# antes de volver a comenzar.
# =====

> "$SALIDA"

# =====
# ARCHIVO TEMPORAL
# =====

TEMP=$(mktemp)

# =====
# EXTRACCIÓN
# =====
# Recorre todos los txt de la carpeta.
# =====

for archivo in "$ORIGEN"/*.txt
do

    cat "$archivo" >> "$TEMP"

    echo "" >> "$TEMP"

done

# =====
# SHUFFLE
# =====
# shuf mezcla líneas aleatoriamente.
#
# Linux posee pequeñas herramientas muy poderosas.

```

```
# =====  
  
shuf "$TEMP" > "$SALIDA"  
  
# =====  
# LIMPIEZA  
# =====  
  
rm "$TEMP"  
  
# =====  
# FEEDBACK  
# =====  
  
echo "Archivo generado:"  
echo "$SALIDA"
```

.gitignore

```
*.class  
.vscode/  
bin/
```

Branches Recomendadas

```
main  
|  
├─ visual-lab  
|   exploración estética  
|  
└─ system-lab  
    arquitectura  
    filesystem  
    sockets  
    networking
```

Primer Commit Recomendado

```
git add .  
git commit -m "Refactor inicial y organización arquitectónica"
```

Ramificación

```
git branch visual-lab  
git branch system-lab
```

Recomendación Técnica Inmediata

Próximo gran objetivo:

```
WatchService
```

Eso permitirá:

```
editar archivo  
↓  
Marquesina detecta cambio  
↓  
recarga automática
```

Ahí el sistema comienza lentamente a abandonar el modo estático y entra al territorio de eventos en tiempo real.